

Docs

Guias de como fazer cada coisa neste projeto.

Guia	Assunto
01-cadastrar.md	Adicionar um registro com ModelForm
02-listar.md	Mostrar todos os registros na tela
03-pesquisar.md	Filtrar a lista por um campo
04-editar.md	Alterar um registro existente
05-excluir.md	Apagar um registro
06-model-e-form.md	Criar campos, choices e o ModelForm
07-urls.md	Ligar uma view a uma rota
08-estilos-static.md	CSS em arquivo separado e collectstatic
09-comandos.md	Comandos do manage.py
10-erros-comuns.md	O que fazer quando quebra

A ideia geral

Toda funcionalidade se resume a duas perguntas:

1. **De onde vem o dado?** `request.POST` (vai mudar o banco) ou `request.GET` (so vai olhar)
2. **O que faco com o model?** `.all()` `.filter()` `.get()` `.save()` `.delete()`

Quero	Uso
tudo	<code>Peso.objects.all()</code>
filtrado	<code>Peso.objects.filter(campo__icontains=x)</code>
um so	<code>Peso.objects.get(id=x)</code>
criar	<code>PesoForm(request.POST)</code> + <code>.save()</code>
editar	<code>PesoForm(request.POST, instance=peso)</code> + <code>.save()</code>
apagar	<code>peso.delete()</code>

Como cadastrar um registro

A view

```
from django.shortcuts import render
from .models import Peso
from .forms import PesoForm

def home(request):
    if request.method == 'POST':
        form = PesoForm(request.POST)      # form com os dados que chegaram
        if form.is_valid():
            form.save()                    # grava no banco
        else:
            form = PesoForm()              # form vazio

    return render(request, 'index.html', {'form': form})
```

O template

```
<form method='post'>
    {% csrf_token %}
    {{ form }}
    <button> salvar </button>
</form>
```

Por que cada parte existe

- `request.method == 'POST'` — a mesma view atende duas situações: abrir a página (GET, mostra o form vazio) e receber o envio (POST, salva). O `if` separa as duas.
- `PesoForm(request.POST)` — sem o `request.POST` dentro, o form nasce vazio e não salva nada. Esse é o erro mais comum.
- `form.is_valid()` — valida os campos. Se estiver inválido, o form volta pro template já com as mensagens de erro dentro dele.
- `form.save()` — como é um `ModelForm`, ele sabe em qual tabela gravar.
- `{% csrf_token %}` — obrigatório em todo form `post`. Sem ele dá erro 403.
- `{{ form }}` — desenha todos os campos de uma vez, com labels.

Se quiser desenhar campo por campo

```
{{ form.nome.label }} {{ form.nome }}  
{{ form.valor.label }} {{ form.valor }}
```

Limpar o form depois de salvar

Do jeito acima, os dados continuam na tela depois de salvar. Para limpar:

```
if form.is_valid():  
    form.save()  
    form = PesoForm()      # troca por um vazio
```

Ou redirecionar (melhor, evita salvar duas vezes se o usuario der F5):

```
from django.shortcuts import redirect  
  
if form.is_valid():  
    form.save()  
    return redirect('/home')
```

Como listar os registros

A view

```
def home(request):
    pesos = Peso.objects.all()
    return render(request, 'index.html', {'pesos': pesos})
```

O dicionario no final do `render` e o **contexto**: e a unica coisa que o template enxerga. A chave (`'pesos'`) e o nome que voce vai usar no HTML.

O template

```
<ul>
    {% for peso in pesos %}
        <li>{{ peso.nome }} - {{ peso.marca }}</li>
    {% empty %}
        <li>Nenhum peso cadastrado.</li>
    {% endfor %}
</ul>
```

- `{% empty %}` roda quando a lista vem vazia. Evita a pagina ficar em branco.
- Todo `{% for %}` precisa de `{% endfor %}`.

Formatando os valores

```
{{ peso.valor|floatformat:2 }}      12.50
{{ peso.date|date:"d/m/Y" }}        23/09/2026
{{ peso.get_tipo_display }}         Halter   (em vez de HAL)
```

O `|` e um **filtro**: transforma o valor antes de mostrar. O `:` passa um argumento.

`get_<campo>_display` so funciona em campo que tem `choices`. Sem ele aparece o codigo cru que esta salvo no banco.

Ordenando

```
Peso.objects.all().order_by('nome')    # A -> Z
Peso.objects.all().order_by('-valor')   # maior -> menor (o menos inverte)
```

Contando

```
{{ pesos|length }} pesos cadastrados
```

Erro classico

Se a lista aparece vazia mesmo tendo dados, quase sempre o nome no contexto (`{'pesos': ...}`) esta diferente do nome no `{% for %}` . Eles tem que bater.

Como pesquisar

E a listagem com `.filter()` no lugar do `.all()`.

A view

```
def home(request):
    busca = request.GET.get('busca')          # le da URL: /home?busca=halter

    if busca:
        pesos = Peso.objects.filter(nome__icontains=busca)
    else:
        pesos = Peso.objects.all()

    return render(request, 'index.html', {'pesos': pesos, 'busca': busca})
```

O template

```
<form method='get'>
    <input type="text" name="busca" value="{{ busca|default_if_none:'' }}" placeholder="Pesquisar" />
    <button> pesquisar </button>
</form>
```

Por que cada parte existe

- `method='get'` — a busca so le, nao muda o banco. Por isso vira parte da URL (`/home?busca=halter`) e nao precisa de `{% csrf_token %}`. O form de cadastro e `post` porque grava.
- `name="busca"` no input e `.get('busca')` na view — essa string igual nos dois lugares e a unica ligacao entre o HTML e o Python. Se errar uma letra, nao funciona.
- `if busca:` — quando ninguem pesquisou, `.get()` devolve `None`; ai cai no `else` e mostra tudo.
- `value="{{ busca }}"` — faz o texto continuar escrito na caixa depois de pesquisar. Sem isso a caixa volta vazia e fica estranho.
- `|default_if_none:''` — sem ele, apareceria a palavra `None` dentro da caixa quando a pagina abre sem busca. Alternativa: usar `request.GET.get('busca', '')` na view, ai o segundo argumento ja e o padrao e o template fica so `{{ busca }}`.

Tipos de filtro

O padrao e `campo__comparacao=valor`:

nome__icontains='hal'	contem, ignorando maiuscula -> LIKE %hal%
nome__contains='Hal'	contem, respeitando maiuscula
nome__exact='Halter'	igual exato
nome__startswith='Hal'	comeca com
valor__gte=100	maior ou igual
valor__lte=100	menor ou igual
date__year=2026	ano da data
tipo='HAL'	sem __ nenhum = igual

Filtrando por mais de um campo

Virgula significa E:

```
Peso.objects.filter(nome__icontains=busca, tipo='HAL')
```

Para OU, precisa do `Q`:

```
from django.db.models import Q
Peso.objects.filter(Q(nome__icontains=busca) | Q(marca__icontains=busca))
```

Busca em dois campos ao mesmo tempo

Util quando o usuario digita e nao sabe se e nome ou marca — use o `Q` acima.

Juntando busca e ordenacao

```
pesos = Peso.objects.filter(nome__icontains=busca).order_by('nome')
```

Pode encadear a vontade: o Django so vai no banco quando o resultado e usado.

Como editar um registro

E o cadastro com uma palavra a mais: `instance`.

A view

```
from django.shortcuts import render, redirect

def editar(request, id):
    peso = Peso.objects.get(id=id)          # pega o registro pelo id da URL

    if request.method == 'POST':
        form = PesoForm(request.POST, instance=peso)    # sem instance criaria um NOVO
        if form.is_valid():
            form.save()
            return redirect('/home')
    else:
        form = PesoForm(instance=peso)          # abre ja preenchido

    return render(request, 'editar.html', {'form': form})
```

A rota

```
path('editar/<int:id>', views.editar)
```

O `<int:id>` captura um numero da URL e entrega como argumento pra view. O nome dentro dos `<>` tem que ser igual ao parametro da funcao (`def editar(request, id)`).

O link na listagem

```
<a href="/editar/{{ peso.id }}">editar</a>
```

Todo registro tem um `id` automatico, criado pelo Django. Voce nao declara no model.

O template editar.html

Igual ao de cadastro:

```
<form method='post'>
    {% csrf_token %}
    {{ form }}
    <button> salvar </button>
</form>
```

A unica coisa pra decorar

Sem instance	Com instance
<code>PesoForm(request.POST)</code>	<code>PesoForm(request.POST, instance=peso)</code>
cria um registro novo	altera o registro existente

`instance` no GET (`PesoForm(instance=peso)`) serve pra outra coisa: preencher os campos com o que ja esta salvo, pra pessoa ver o que esta editando.

Usando a mesma view pra criar e editar

Da pra fazer o `id` ser opcional:

```
def salvar(request, id=None):
    peso = Peso.objects.get(id=id) if id else None
    form = PesoForm(request.POST or None, instance=peso)
    if form.is_valid():
        form.save()
        return redirect('/home')
    return render(request, 'editar.html', {'form': form})
```

Mais curto, mas mais dificil de ler. Para a prova, duas views separadas e mais seguro.

Como excluir um registro

A view

```
from django.shortcuts import redirect

def excluir(request, id):
    peso = Peso.objects.get(id=id)
    peso.delete()
    return redirect('/home')
```

Tres linhas: acha, apaga, volta pra listagem.

A rota

```
path('excluir/<int:id>', views.excluir)
```

O link

```
<a href="/excluir/{{ peso.id }}">excluir</a>
```

Por que o redirect

Sem ele a view nao devolve nada e da erro (`didn't return an HttpResponse object`). Depois de apagar nao ha o que mostrar, entao manda a pessoa de volta pra lista.

Pedindo confirmacao (opcional)

Com um `confirm` do JavaScript, sem pagina nova:

```
<a href="/excluir/{{ peso.id }}" onclick="return confirm('Excluir esse peso?')">excluir</a>
```

Ou com uma pagina de confirmacao, que e o jeito mais correto:

```
def excluir(request, id):
    peso = Peso.objects.get(id=id)

    if request.method == 'POST':
        peso.delete()
        return redirect('/home')

    return render(request, 'confirmar.html', {'peso': peso})
```

```
<p>Excluir {{ peso.nome }}?</p>
<form method='post'>
    {% csrf_token %}
    <button> sim, excluir </button>
</form>
```

Por que isso é melhor: apagar por link (`get`) significa que qualquer coisa que abra a URL apaga o registro. Por `post` , so apaga quem clicou no botão de verdade.

Como criar o model e o form

O model (pesos/models.py)

```
from django.db import models

class Peso(models.Model):
    class Tipo(models.TextChoices):
        HALTER = 'HAL', 'Halter'          # valor no banco, texto na tela
        ANILHA = 'ANI', 'Anilha'
        BARRA = 'BAR', 'Barra'

    nome = models.CharField(max_length=100)
    marca = models.CharField(max_length=100)
    valor = models.FloatField()
    date = models.DateField()
    tipo = models.CharField(
        max_length=3,
        choices=Tipo.choices,
        default=Tipo.HALTER,
    )

    def __str__(self):
        return self.nome                # como o registro aparece no admin e no shell
```

Cada classe e uma tabela. Cada atributo e uma coluna. O `id` o Django cria sozinho.

Campos mais usados

<code>models.CharField(max_length=100)</code>	texto curto (max_length e obrigatorio)
<code>models.TextField()</code>	texto longo
<code>models.IntegerField()</code>	numero inteiro
<code>models.FloatField()</code>	numero com virgula
<code>models.DecimalField(max_digits=8, decimal_places=2)</code>	dinheiro (mais correto)
<code>models.DateField()</code>	data
<code>models.DateTimeField()</code>	data + hora
<code>models.BooleanField()</code>	sim/nao
<code>models.EmailField()</code>	texto com validacao de email
<code>models.ImageField(upload_to='fotos/')</code>	imagem

Argumentos que servem em qualquer campo:

```
null=True          aceita vazio no banco
blank=True         aceita vazio no formulario
default='x'        valor inicial
choices=...        lista fechada de opcoes
verbose_name=''    nome que aparece na label
```

Choices

`TextChoices` faz o campo virar um `<select>` no form automaticamente.

```
HALTER = 'HAL', 'Halter'
#      ^^^^^ ^^^^^^^
#      banco  tela
```

No template, para mostrar o texto bonito:

```
{{ peso.get_tipo_display }}      Halter
{{ peso.tipo }}                  HAL
```

Depois de mexer no model, SEMPRE

```
python manage.py makemigrations
python manage.py migrate
```

`makemigrations` escreve o arquivo com as instrucoes. `migrate` executa no banco. Esquecer isso da o erro `no such table` ou `no such column`.

O form (pesos/forms.py)

```
from django import forms
from .models import Peso

class PesoForm(forms.ModelForm):
    class Meta:
        model = Peso
        fields = ['nome', 'valor', 'marca', 'date', 'tipo']
```

`ModelForm` le o model e monta os campos sozinho — nao precisa redeclarar nada.

- `fields = '__all__'` inclui todos
- `exclude = ['date']` inclui todos menos esses
- A ordem da lista e a ordem na tela

Deixando o campo de data com calendario

Por padrao o `DateField` vira uma caixa de texto onde a pessoa tem que digitar `2026-09-23` . Para virar um seletor de data do navegador:

```
class PesoForm(forms.ModelForm):
    class Meta:
        model = Peso
        fields = ['nome', 'valor', 'marca', 'date', 'tipo']
        widgets = {
            'date': forms.DateInput(attrs={'type': 'date'}),
        }
```

`widgets` troca o HTML que o campo gera. Da pra usar o mesmo truque pra colocar classe de CSS ou placeholder:

```
widgets = {
    'nome': forms.TextInput(attrs={'placeholder': 'Ex: Halter 10kg', 'class': 'input'}),
}
```

Registrando no admin (pesos/admin.py)

```
from django.contrib import admin
from pesos.models import Peso

admin.site.register(Peso)
```

Sem essa linha o model nao aparece em `/admin/` .

Como ligar uma view a uma rota

Sao dois arquivos: o do app e o do projeto.

1. No app (pesos/urls.py)

```
from . import views
from django.urls import path

urls = [
    path('home', views.home),
    path('editar/<int:id>', views.editar),
    path('excluir/<int:id>', views.excluir),
]
```

2. No projeto (academia/urls.py)

```
from django.contrib import admin
from django.urls import path
from pesos import urls as pesosViews

urlpatterns = [
    path('admin/', admin.site.urls),
] + pesosViews.urls
```

Aqui a lista do app e somada na lista do projeto. Esse e o jeito que este projeto usa. O jeito mais comum na documentacao e com `include`:

```
# no projeto
from django.urls import include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('pesos.urls')),
]

# no app, a lista precisa se chamar urlpatterns
urlpatterns = [
    path('home', views.home),
]
```

Os dois funcionam. Nao misture.

Passando valor pela URL

```
path('editar/<int:id>', views.editar)
```

```
def editar(request, id):    # o nome tem que ser igual ao de dentro dos <>
```

Tipos: `<int:id>` numero, `<str:nome>` texto, `<slug:slug>` texto-com-hifen.

Dando nome pra rota

```
path('home', views.home, name='home')
```

Ai no template voce usa o nome em vez do caminho:

```
<a href="{% url 'home' %}">inicio</a>
<a href="{% url 'editar' peso.id %}">editar</a>
```

Vantagem: se um dia mudar o caminho de `/home` pra `/inicio`, os links continuam funcionando. Se usar `{% url %}` sem ter posto `name=`, da `NoReverseMatch`.

A rota vazia (pagina inicial)

```
path('', views.home)    # responde em http://127.0.0.1:8000/
```

Como colocar CSS (arquivos estaticos)

Hoje o CSS deste projeto esta dentro de uma tag `<style>` no proprio `index.html`. Funciona, mas o certo e por num arquivo `.css` separado. Isso e um **arquivo estatico** (static file): CSS, JavaScript, imagens, fontes — tudo que o navegador baixa pronto, sem o Django gerar nada.

1. Criar a pasta

Dentro do app, seguindo o padrao `app/static/app/`:

```
pesos/  
  static/  
    pesos/  
      style.css
```

Por que repetir o nome do app: todos os apps jogam os estaticos no mesmo lugar no final. Se dois apps tiverem um `style.css` solto, um sobrescreve o outro. A subpasta com o nome do app evita a colisao.

2. Usar no template

```
{% load static %}  
<!DOCTYPE html>  
<html>  
<head>  
  <link rel="stylesheet" href="{% static 'pesos/style.css' %}">  
</head>
```

- `{% load static %}` vai na **primeira linha** do arquivo, antes do `<!DOCTYPE>`
- `{% static '...' %}` monta a URL certa. Nunca escreva o caminho na mao.

3. Conferir o settings.py

Ja vem assim por padrao:

```
STATIC_URL = 'static/'
```

E o `django.contrib.staticfiles` precisa estar no `INSTALLED_APPS` (tambem ja vem).

Com isso **em desenvolvimento ja funciona**: `runserver` acha os arquivos sozinho.

E o collectstatic?

`collectstatic` **nao e necessario pra desenvolver**. Ele existe pra producao.

Em desenvolvimento (`DEBUG = True`), o `runserver` sai procurando os estaticos em cada app, um por um, na hora do pedido. Isso e lento e o servidor de verdade (nginx, Apache) nao sabe fazer isso. Entao, antes de publicar, voce roda um comando que **copia todos os estaticos de todos os apps pra uma pasta unica**:

```
# settings.py
import os

STATIC_URL = 'static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')    # pasta de destino
```

```
python manage.py collectstatic
```

Resultado:

```
staticfiles/          <- criada pelo comando, e so copia
  admin/              <- o CSS do admin do Django veio junto
  pesos/
    style.css
```

Regras pra nao se perder:

- `STATIC_ROOT` e **destino**, nunca onde voce escreve. Nao edite nada la dentro — o proximo `collectstatic` sobrescreve.
- `STATIC_ROOT` nao pode ser igual a nenhuma pasta `static/` sua, senao da erro.
- A pasta `staticfiles/` vai pro `.gitignore` — e gerada, nao e codigo.
- Rode de novo toda vez que mudar um CSS em producao.

STATICFILES_DIRS (estaticos fora dos apps)

Se quiser uma pasta `static/` na raiz do projeto, valendo pro site inteiro em vez de um app so:

```
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]
```

O `collectstatic` tambem recolhe dessas pastas.

Resumo dos tres nomes parecidos

Nome	O que e
STATIC_URL	o prefixo na URL: <code>/static/pesos/style.css</code>
STATICFILES_DIRS	pastas extras de onde ler , fora dos apps
STATIC_ROOT	pasta unica pra onde copiar , so usada pelo collectstatic

Imagem no template

```

```

Mesma coisa: `{% load static %}` no topo e `{% static %}` no caminho.

Cuidado: `static` (CSS/JS/imagens do site) e diferente de `media` (arquivos que o usuario envia por upload). `MEDIA_URL` / `MEDIA_ROOT` sao outra configuracao e o `collectstatic` nao mexe neles.

Comandos

Do dia a dia

```
python manage.py runserver          # sobe o servidor em 127.0.0.1:8000
python manage.py runserver 8080     # em outra porta
python manage.py makemigrations     # gera o arquivo de migracao depois de mexer no model
python manage.py migrate            # aplica no banco
python manage.py createsuperuser    # cria o login do /admin
python manage.py check              # procura erro de configuracao sem subir o servidor
```

Para parar o servidor: `Ctrl + C`.

Comecando um projeto do zero

```
python -m venv venv
venv\Scripts\activate          # Windows
pip install django

django-admin startproject academia .    # o ponto evita criar pasta duplicada
python manage.py startapp pesos
```

Depois do `startapp`, registre o app no `settings.py`:

```
INSTALLED_APPS = [
    ...
    'pesos',
]
```

Esquecer isso e a causa numero 1 de `TemplateDoesNotExist`.

Rodando este projeto

```
venv\Scripts\activate
python manage.py migrate          # cria o banco, que nao vem no repositario
python manage.py runserver
```

Abre em `http://127.0.0.1:8000/home`

Ambiente virtual

```
python -m venv venv          # cria
venv\Scripts\activate       # ativa (aparece (venv) no terminal)
deactivate                  # sai
pip freeze > requirements.txt # salva a lista de pacotes
pip install -r requirements.txt # instala a lista em outra maquina
```

Migracoes quando da errado

```
python manage.py showmigrations # ve o que ja foi aplicado
python manage.py makemigrations pesos # so de um app
python manage.py migrate pesos 0001 # volta pra uma migracao anterior
```

Em projeto de estudo, quando o banco embola de vez, o caminho rapido e apagar o `db.sqlite3` e os arquivos numerados de `pesos/migrations/` (menos o `__init__.py`) e rodar `makemigrations` + `migrate` de novo. Isso **apaga todos os dados** — so faca em projeto de aula.

Shell (testar consultas sem navegador)

```
python manage.py shell
```

```
from pesos.models import Peso
Peso.objects.all()
Peso.objects.filter(nome__icontains='hal')
Peso.objects.create(nome='Halter 10kg', marca='X', valor=90, date='2026-09-23', tipo='HAL')
```

Otimo pra conferir se o filtro que voce escreveu na view devolve o que voce espera.

Erros comuns

Erro na tela	Causa mais provavel	Conserto
TemplateDoesNotExist	app fora do <code>INSTALLED_APPS</code> , ou template fora de <code>app/templates/</code>	registrar o app / mover o arquivo
CSRF verification failed	faltou <code>{% csrf_token %}</code> num form post	por a tag dentro do <code><form></code>
no such table: pesos_peso	faltou rodar as migracoes	<code>makemigrations</code> + <code>migrate</code>
no such column	mexeu no model e nao migrou	<code>makemigrations</code> + <code>migrate</code>
NoReverseMatch	usou <code>{% url 'x' %}</code> sem <code>name='x'</code> na rota	por o <code>name=</code> no <code>path()</code>
didn't return an HttpResponse	a view nao tem <code>return render(...)</code> ou <code>redirect(...)</code>	devolver alguma resposta
Peso matching query does not exist	<code>.get(id=...)</code> com id que nao existe	conferir o link, ou usar <code>get_object_or_404</code>
Page not found (404)	a URL digitada nao bate com nenhum <code>path()</code>	conferir <code>urls.py</code> (este projeto usa <code>/home</code> , sem barra no fim)
Port is already in use	ja tem um servidor rodando	<code>Ctrl+C</code> no outro terminal, ou <code>runserver 8080</code>

Erros que nao mostram erro nenhum

Esses sao os piores, porque a pagina abre normal e so nao faz o que deveria.

O form nao salva e nao reclama

Quase sempre e o `request.POST` que faltou:

```
form = PesoForm()           # errado – nasce vazio, is_valid() da False
form = PesoForm(request.POST) # certo
```

Se ainda assim nao salvar, e porque falta o `.save()` dentro do `if form.is_valid():` , ou os dados estao invalidos. Para ver o motivo:

```
print(form.errors)
```

Aparece no terminal onde o `runserver` esta rodando.

A lista aparece vazia mesmo tendo dados

O nome no contexto esta diferente do nome no `{% for %}`:

```
return render(request, 'index.html', {'pesos': pesos})
```

```
{% for peso in pesos %}      <- tem que ser 'pesos', igual a chave
```

O choice mostra codigo em vez do texto

```
{{ peso.tipo }}              HAL  
{{ peso.get_tipo_display }}  Halter   <- este
```

O CSS nao carrega

Faltou `{% load static %}` na primeira linha do template, ou o caminho dentro do `{% static %}` esta errado. Confira tambem se o arquivo esta em `app/static/app/`.

Mudei o CSS e nao muda nada

Cache do navegador. `Ctrl + F5`.

Como ler um erro do Django

A pagina amarela de erro tem tudo. Duas coisas resolvem 90%:

1. A **primeira linha em negrito** no topo diz o que aconteceu
2. No meio da pilha, procure a linha que aponta pro **seu** arquivo (`pesos/views.py`), nao pros arquivos de dentro do `django/`. O erro esta la.